

Qt Base Course

Day 1

Fundamentals & Basic GUI Techniques

Day 1 — Agenda

Fundamentals

- What is Qt? — editions, licensing, the Qt ecosystem
- IDE integration and Qt Creator
- Signal-slot mechanism and the Meta-Object Compiler
- `Q_OBJECT` , `connect()` , and the three syntax styles
- Object management trees — automatic memory management
- Multilingual support and localisation

Basic GUI Techniques

- `QWidget` class and widget nesting
- GUI programming with Qt Designer
- `QPainter` and custom drawings
- 2D transformations
- `QMainWindow` and `QDialog`
- Layout management

Part 1 — Fundamentals

What is Qt?

- A **cross-platform application framework** written in C++ — one codebase, many targets
- Originally developed by Trolltech (1991), now maintained by **The Qt Company**
- Covers GUI, networking, databases, threads, multimedia, 3D, QML, and more
- Used in automotive, industrial, medical, embedded, and desktop software worldwide

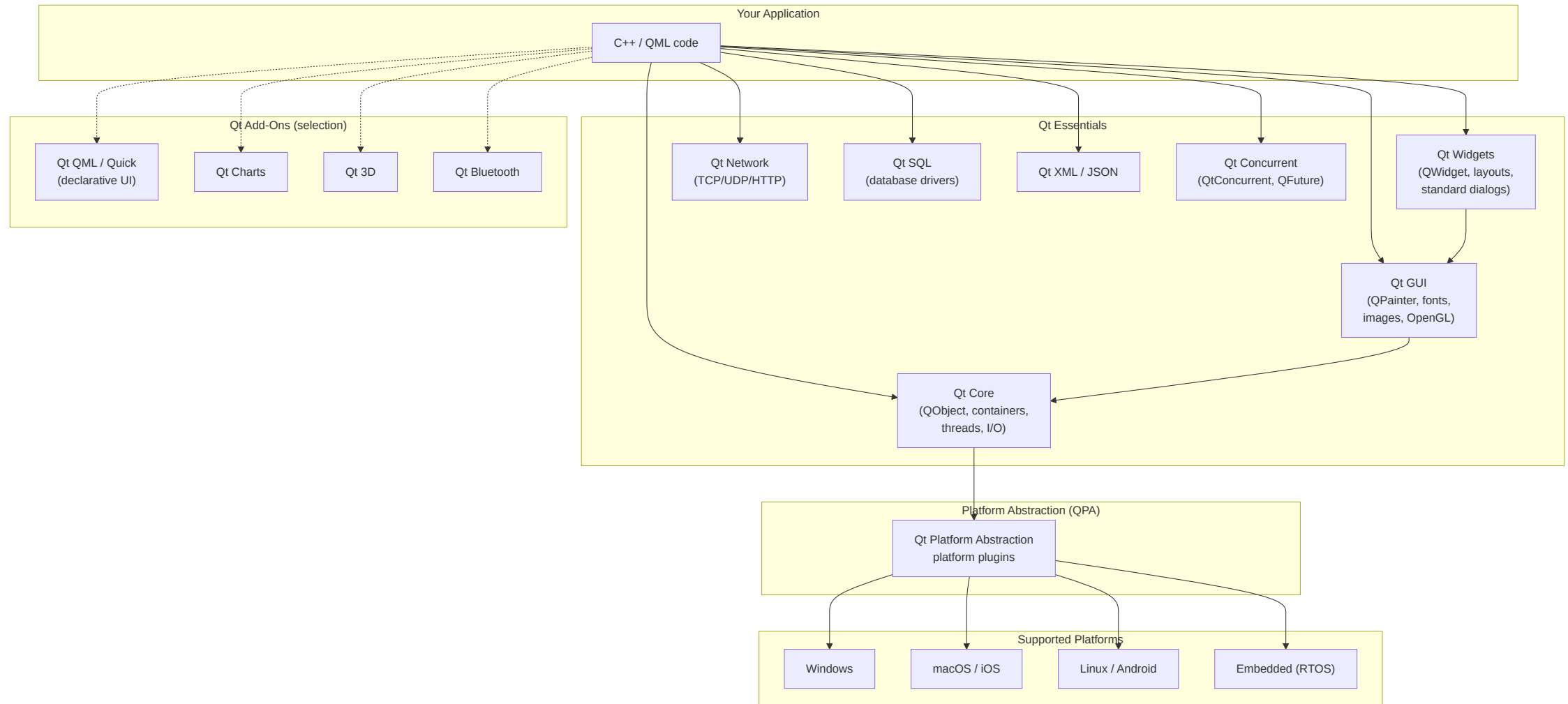
"Write once, compile everywhere" — Windows, macOS, Linux, Android, iOS, and embedded RTOS

Qt Editions and Licensing

Edition	Licence	Target
Qt Open Source	LGPL 3 / GPL 3	Open-source projects
Qt Commercial	Commercial	Proprietary software, no LGPL obligations
Qt for Device Creation	Commercial	Embedded / IoT products

- LGPL allows use in closed-source apps **if** Qt is dynamically linked and you don't modify Qt itself
- When in doubt: use the commercial licence for proprietary products

Qt Architecture



Key Qt Modules

Module	Highlights
Qt Core	<code>QObject</code> , containers, I/O, threads, timers
Qt GUI	<code>QPainter</code> , fonts, images, OpenGL surface
Qt Widgets	<code>QWidget</code> , standard dialogs, layouts
Qt Network	TCP, UDP, HTTP, SSL
Qt SQL	DB drivers, <code> QSqlQuery </code> , SQL models
Qt Concurrent	<code>QtConcurrent::run</code> , <code>QFuture</code>
Qt QML / Quick	Declarative UI (not covered in this course)

IDE Integration and Qt Creator

Qt Creator is the official Qt IDE:

- **Projects** — CMake (preferred in Qt 6), qmake, Qbs
- **Editor** — C++ and QML with autocompletion, refactoring
- **Designer** — drag-and-drop form editor built in
- **Debugger** — GDB / LLDB integration with pretty-printers for Qt types
- **Profiler** — CPU, memory, QML profiling
- **Device Manager** — deploy and run on remote targets (Raspberry Pi, Android, etc.)

Qt Creator — Project Layout (CMake)

```
MyApp/
├── CMakeLists.txt           ← project definition
├── main.cpp
├── mainwindow.h
├── mainwindow.cpp
├── mainwindow.ui           ← Qt Designer form
└── resources.qrc           ← embedded resources (icons, images)
```

```
cmake_minimum_required(VERSION 3.16)
project(MyApp LANGUAGES CXX)

find_package(Qt6 REQUIRED COMPONENTS Core Gui Widgets)

qt_add_executable(MyApp main.cpp mainwindow.cpp)
target_link_libraries(MyApp PRIVATE Qt6::Core Qt6::Gui Qt6::Widgets)
```

Part 2 — Signal-Slot & Meta-Object Compiler

The Problem: Loose Coupling in GUI

Classic callback approach — tight coupling, unsafe function pointers:

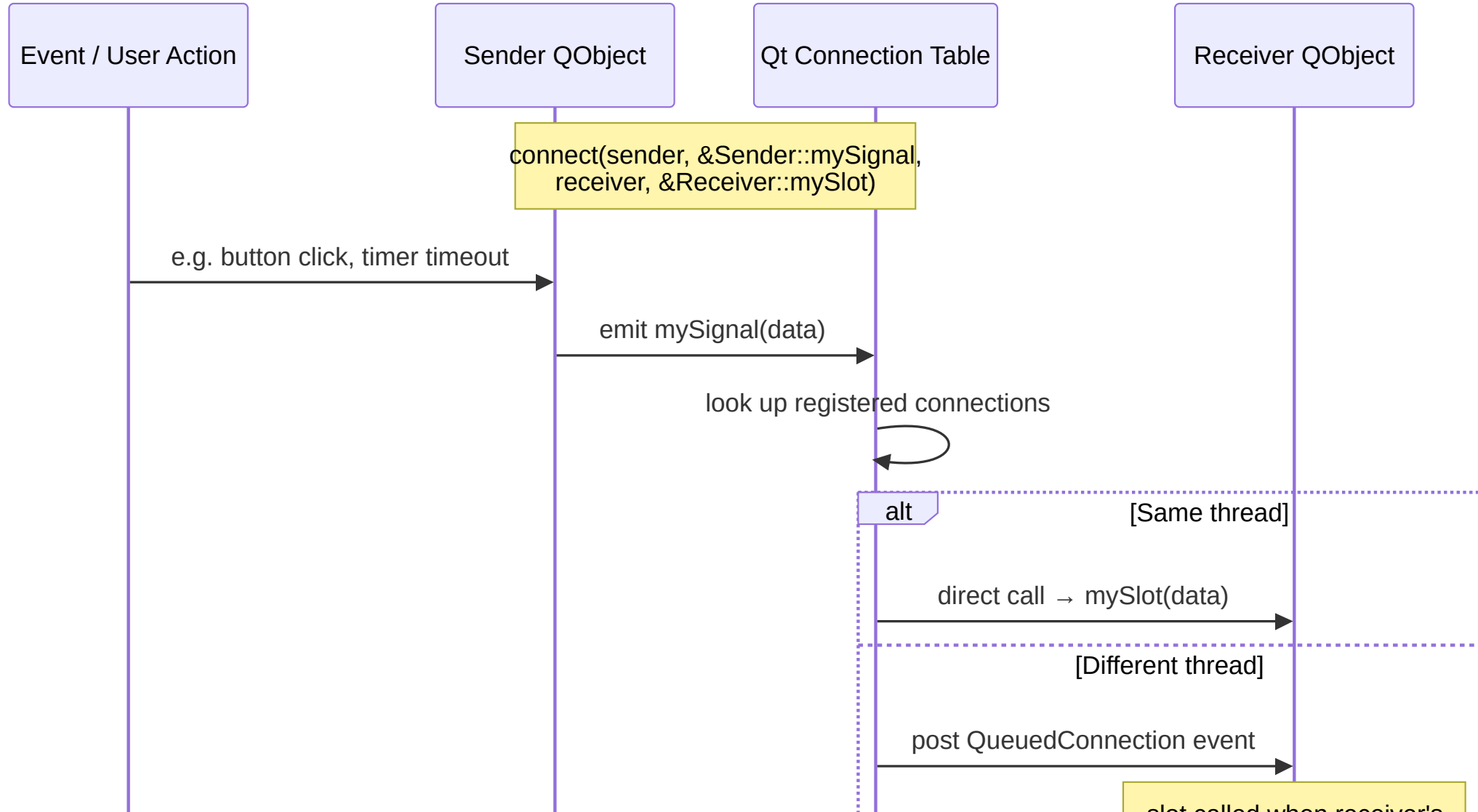
```
// C-style callback – no type safety, no object context  
button->setCallback(&onClicked); // which object? which method?
```

Qt's answer: **signals and slots** — a type-safe observer pattern built into the language:

```
connect(button, &QPushButton::clicked,  
        this,   &MainWindow::onButtonClicked);
```

- The sender does not know who listens
- Multiple receivers can connect to one signal
- One receiver can connect to multiple signals

Signals and Slots — The Concept



`connect()` — Old-Style (String-Based)

```
// Qt 4 / Qt 5 compatible – runtime checked, no compile-time type safety
connect(button, SIGNAL(clicked(bool)),
        this,   SLOT(onClicked(bool)));
```

- Errors only discovered at **runtime** (message in debug output)
- Still works in Qt 6 but not recommended for new code

`connect()` — New-Style (Function Pointers)

```
// Qt 5+ – compile-time type checking
connect(button, &QPushButton::clicked,
        this,   &MainWindow::onClicked);
```

- Compiler catches mismatched signatures immediately
- Supports **overloaded** signals with explicit cast:

```
connect(spinBox,
        QOverload<int>::of(&QSpinBox::valueChanged),
        this,
        &MainWindow::onValueChanged);
```

connect() — Lambda Slots

```
// Capture context, inline logic – great for short handlers
connect(button, &QPushButton::clicked, this, [this]() {
    label->setText("Clicked!");
    statusBar()->showMessage("Done", 2000);
});
```

- Pass `this` as context object so the connection is removed when `this` is destroyed
- Avoid heavy logic in lambdas — prefer named slots for testability

The Meta-Object Compiler (MOC)

Qt extends C++ with features the language doesn't natively support:

Feature	How
Signals and slots	<code>Q_OBJECT</code> + MOC generates connection code
<code>QObject::connect</code> runtime introspection	MOC stores method signatures
<code>qobject_cast<T*></code> — safe downcasting	MOC generates type info
Property system	<code>Q_PROPERTY</code> macro processed by MOC
Translation — <code>tr()</code>	MOC registers the class name

MOC is a code generator — it reads your header and produces `moc_myclass.cpp` at

The `Q_OBJECT` Macro

Any class that uses signals, slots, or properties **must** have `Q_OBJECT` in its private section:

```
class MyWidget : public QWidget {
    Q_OBJECT // ← must be first in private section

public:
    explicit MyWidget(QWidget *parent = nullptr);

signals:
    void valueChanged(int newValue);

public slots:
    void setValue(int value);

private:
    int m_value = 0;
};
```

Defining Signals and Slots

```
// mywidget.cpp – slot implementation (signals have no implementation)
void MyWidget::setValue(int value) {
    if (m_value == value)
        return;                // avoid infinite loops when cross-connected
    m_value = value;
    emit valueChanged(value);    // emit is just a macro – expands to nothing
}
```

- **Signals** are declared, never defined — MOC generates the body
- **Slots** are ordinary C++ functions — call them directly if needed
- `emit` is optional but documents intent clearly

Part 3 — Object Management Trees

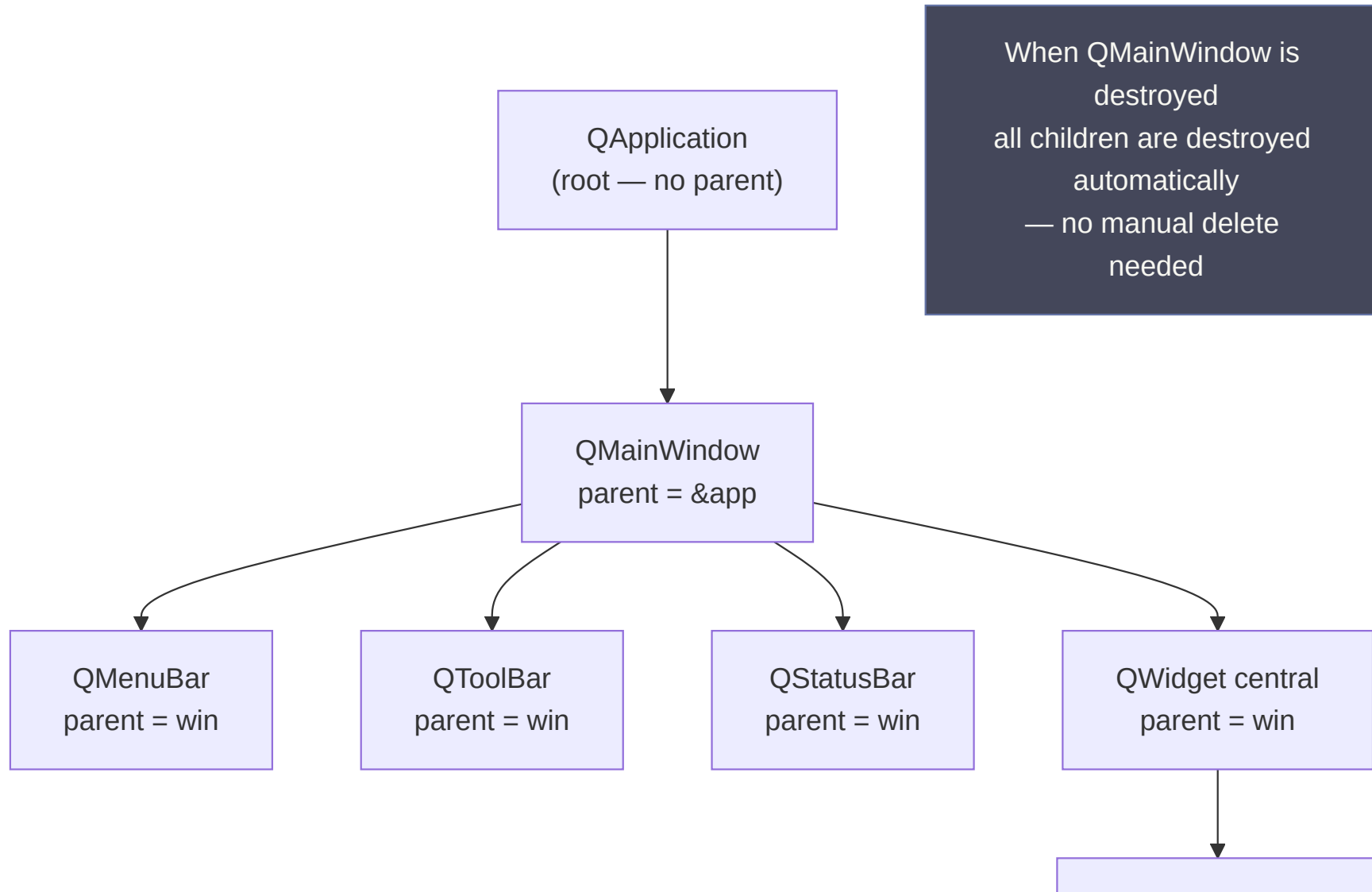
QObject: The Root of (Almost) Everything

`QObject` is the base class for nearly all Qt classes:

- Provides signals, slots, properties, and event handling
- Implements **parent-child ownership**
- Enables `QObject::cast`, `QObject::findChild`, `QObject::findChildren`
- Not copyable — no copy constructor or assignment operator

```
QObject *parent = new QObject;  
QObject *child  = new QObject(parent); // child's lifetime tied to parent  
  
delete parent; // also deletes child – no leak
```

Object Tree



Parent-Child Ownership Rules

Rule	Detail
A child is deleted when its parent is deleted	Recursive — whole subtree cleaned up
A child can be reparented	<code>child->setParent(newParent)</code>
A child can be removed from parent	<code>child->setParent(nullptr)</code> — now you own it
Stack-allocated children are safe	Parent does not double-delete them
Pass <code>this</code> as parent in widget constructors	Standard Qt idiom

```
// Typical widget constructor
MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent) {
    // ...
    child->setParent(this);
}
```

findChild and findChildren

```
// Find a single named child (any depth)
auto *btn = window->findChild<QPushButton *>("submitButton");

// Find all QLabel descendants
const auto labels = window->findChildren<QLabel *>();
for (auto *lbl : labels)
    lbl->setEnabled(false);
```

Useful for programmatically accessing widgets loaded from `.ui` files.

Part 4 — Multilingual Support & Localisation

Marking Strings for Translation

```
// In any QObject subclass – use tr()  
label->setText(tr("Hello, world!"));  
  
// Outside a QObject – use QObject::tr() or QApplication::translate()  
QString msg = QObject::tr("File not found.");  
  
// Disambiguation hint (same English, different meaning)  
tr("Open", "menu action");  
tr("Open", "button label");
```

Every user-visible string that should be translated must be wrapped in `tr()`.

The Localisation Workflow

lupdate MyApp.pro	← 1. extract tr() strings into .ts files
Qt Linguist (GUI tool)	← 2. translator fills in translations
lrelease MyApp.pro	← 3. compile .ts → .qm (binary format)
QTranslator at runtime	← 4. load .qm and install into QApplication

```
// main.cpp – load translation at startup
QTranslator translator;
if (translator.load("myapp_de", ":/translations/"))
    QApplication::installTranslator(&translator);
// All tr() calls now return German strings
```

.ts File Structure

```
<!-- myapp_de.ts (generated by lupdate, edited in Qt Linguist) -->
<TS language="de_DE">
  <context>
    <name>MainWindow</name>
    <message>
      <source>Hello, world!</source>
      <translation>Hallo, Welt!</translation>
    </message>
  </context>
</TS>
```

Add **.ts** files to **CMakeLists.txt** :

```
qt_add_translations(MyApp TS_FILES myapp_de.ts myapp_fr.ts)
```

Part 5 — Basic GUI: QWidget

QWidget — The Base Class

Every visible element in a Qt Widgets application is a `QWidget` or a subclass:

`QWidget`

- `QAbstractButton` → `QPushButton`, `QCheckBox`, `QRadioButton`
- `QAbstractScrollArea` → `QListView`, `QTableView`, `QTextEdit`
- `QDialog` → `QFileDialog`, `QMessageBox`, `QColorDialog`
- `QFrame` → `QLabel`, `QSplitter`
- `QMainWindow`
- ... hundreds more

- A widget with no parent is a **top-level window**
- A widget with a parent is embedded inside it

Widget Nesting

```
QWidget *window = new QWidget;  
window->setWindowTitle("My App");  
  
QWidget *panel = new QWidget(window); // child of window  
panel->setGeometry(10, 10, 200, 100);  
  
QPushButton *btn = new QPushButton("OK", panel); // child of panel  
btn->setGeometry(60, 30, 80, 30);  
  
window->show();
```

Or use layouts (preferred — see Part 9):

```
auto *layout = new QVBoxLayout(window);  
layout->addWidget(new QLabel("Name:"));  
layout->addWidget(new QLineEdit);  
layout->addWidget(new QPushButton("OK"));
```

Part 6 — Qt Designer

Qt Designer — Visual Form Editing

Qt Designer lets you build widget layouts by drag and drop:

- Drag widgets from the widget palette onto the form
- Set properties (text, size policy, object name) in the property editor
- Define layouts by selecting widgets and pressing Ctrl+H / Ctrl+L
- Add signals/slots connections visually in the signal/slot editor
- Save as `.ui` — an XML description of the form

```
<!-- Generated .ui file (excerpt) -->  
<widget class="QPushButton" name="submitButton">  
  <property name="text">  
    <string>Submit</string>  
  </property>  
</widget>
```

Using a `.ui` File in Code

Qt Creator generates a `ui_mainwindow.h` from `mainwindow.ui` at build time.

Single inheritance approach (recommended):

```
// mainwindow.h
#include "ui_mainwindow.h"

class MainWindow : public QMainWindow {
    Q_OBJECT
public:
    explicit MainWindow(QWidget *parent = nullptr);
private:
    Ui::MainWindow ui;    // embedded – not a pointer
};

// mainwindow.cpp
MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent) {
    ui.setupUi(this);    // creates all widgets
    connect(ui.submitButton, &QPushButton::clicked,
```

Part 7 — QPainter & Custom Drawings

Custom Drawing with `paintEvent`

```
class CircleWidget : public QWidget {
    Q_OBJECT
protected:
    void paintEvent(QPaintEvent *event) override {
        QPainter painter(this);
        painter.setRenderHint(QPainter::Antialiasing);

        // filled ellipse
        painter.setBrush(QColor(80, 250, 123)); // green fill
        painter.setPen(QPen(Qt::black, 2));
        painter.drawEllipse(rect().adjusted(10, 10, -10, -10));

        // text in the centre
        painter.setPen(Qt::black);
        painter.drawText(rect(), Qt::AlignCenter, "Qt!");
    }
};
```

Always draw inside `paintEvent` — never store a `QPainter` as a member

QPainter Drawing Primitives

```
QPainter p(this);

p.drawLine(QPointF(0, 0), QPointF(100, 100));
p.drawRect(QRectF(10, 10, 80, 60));
p.drawEllipse(QRectF(10, 10, 80, 60));
p.drawArc(QRectF(10, 10, 80, 60), 30*16, 120*16); // angles in 1/16°
p.drawPolygon(QPolygonF({ {0,50}, {50,0}, {100,50}, {50,100} }));
p.drawPixmap(0, 0, QPixmap(":/icons/logo.png"));
p.drawText(QPointF(20, 40), "Hello Qt");
```

2D Transformations

```
QPainter p(this);
p.setRenderHint(QPainter::Antialiasing);

// save/restore the coordinate system
p.save();
    p.translate(100, 100);    // move origin
    p.rotate(45);            // degrees clockwise
    p.scale(1.5, 1.5);       // scale up
    p.drawRect(-20, -20, 40, 40); // drawn in transformed coords
p.restore();

// or use QTransform directly
QTransform t;
t.rotate(30).scale(2, 2);
p.setTransform(t);
```

Part 8 — QMainWindow & QDialog

QMainWindow Anatomy

```
QMainWindow
├── QMenuBar      (top)
├── QToolBar      (dockable, multiple allowed)
├── Central Widget ← your main content area
├── QDockWidget   (side panels, dockable / floating)
└── QStatusBar    (bottom)
```

```
MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent) {
    // Central widget – must be set
    auto *central = new QWidget(this);
    setCentralWidget(central);

    // Menu
    auto *fileMenu = menuBar()->addMenu(tr("&File"));
    fileMenu->addAction(tr("&Open..."), this, &MainWindow::openFile,
                       QKeySequence::Open);
}
```


QDialog — Modal and Modeless

```
// Modal – blocks the parent window until closed
QDialog dlg(this);
dlg.setWindowTitle(tr("Settings"));
auto *layout = new QFormLayout(&dlg);
auto *nameEdit = new QLineEdit;
layout->addRow(tr("Name:"), nameEdit);
auto *buttons = new QDialogButtonBox(
    QDialogButtonBox::Ok | QDialogButtonBox::Cancel, &dlg);
connect(buttons, &QDialogButtonBox::accepted, &dlg, &QDialog::accept);
connect(buttons, &QDialogButtonBox::rejected, &dlg, &QDialog::reject);
layout->addRow(buttons);

if (dlg.exec() == QDialog::Accepted)
    qDebug() << "User entered:" << nameEdit->text();
```

Standard Dialogs

```
// File picker
QString path = QFileDialog::getOpenFileName(
    this, tr("Open Image"), QDir::homePath(),
    tr("Images (*.png *.jpg *.bmp)"));

// Message box
int ret = QMessageBox::question(this, tr("Confirm"),
                                tr("Delete this file?"),
                                QMessageBox::Yes | QMessageBox::No);

// Input
bool ok;
QString name = QInputDialog::getText(this, tr("Name"),
                                     tr("Enter your name:"),
                                     QLineEdit::Normal, "", &ok);

// Colour picker
QColor colour = QColorDialog::getColor(Qt::blue, this);
```

Part 9 — Layout Management

Why Use Layouts?

Without layouts — fragile hard-coded coordinates:

```
label->setGeometry(10, 10, 80, 25);    // breaks on resize and HiDPI  
edit->setGeometry(100, 10, 200, 25);
```

With layouts — adaptive, DPI-aware, resize-safe:

```
auto *layout = new QHBoxLayout;  
layout->addWidget(label);  
layout->addWidget(edit);
```

The Four Core Layouts

Layout	Behaviour
<code>QHBoxLayout</code>	Horizontal row of widgets
<code>QVBoxLayout</code>	Vertical column of widgets
<code>QGridLayout</code>	Grid of rows and columns
<code>QFormLayout</code>	Two-column label + field pairs

Nesting Layouts

```
auto *mainLayout    = new QVBoxLayout(this);    // outer vertical

auto *topRow        = new QHBoxLayout;
topRow->addWidget(new QLabel(tr("Name:")));
topRow->addWidget(new QLineEdit);

auto *grid          = new QGridLayout;
grid->addWidget(new QCheckBox(tr("Option A")), 0, 0);
grid->addWidget(new QCheckBox(tr("Option B")), 0, 1);
grid->addWidget(new QCheckBox(tr("Option C")), 1, 0);

auto *buttons       = new QHBoxLayout;
buttons->addStretch();                                // pushes buttons right
buttons->addWidget(new QPushButton(tr("OK")));
buttons->addWidget(new QPushButton(tr("Cancel")));

mainLayout->addLayout(topRow);
mainLayout->addLayout(grid);
mainLayout->addStretch();                            // fills remaining space
mainLayout->addLayout(buttons);
```

Stretch Factors and Size Policies

```
// Give the text editor 3× as much horizontal space as the sidebar
splitterLayout->addWidget(sidebar, 1); // stretch = 1
splitterLayout->addWidget(textEditor, 3); // stretch = 3

// Size policy controls how a widget responds to extra space
edit->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Fixed);
// Expanding: takes as much width as possible
// Fixed: height stays at sizeHint()
```

Policy	Meaning
Fixed	Always exactly <code>sizeHint()</code>
Minimum	Can grow, never smaller than hint
Expanding	Should get as much space as possible
Preferred	Uses hint but can shrink or grow